

From RAG to a Tool-Using Retrieval Agent

Build and benchmark three progressively capable retrieval systems on a question-answering dataset: a baseline RAG pipeline, an improved retrieval system with a fine-tuned embedder, and an iterative-retrieval agent.

You will produce **three system configurations** that you evaluate side by side:

Config A - Baseline RAG: off-the-shelf embedder, naive top- k retrieval (Task 2)

Config B - Improved Retrieval: fine-tuned embedder + one architectural technique (Task 3)

Config C - Agent: iterative-retrieval agent built on top of Config B (Task 4)

Task 1. Benchmark & Corpus Selection

Choose one question-answering benchmark from the list below. Multi-hop benchmarks are recommended because the iterative-retrieval agent in Task 4 is most informative on questions that structurally require more than one retrieval step.

Benchmark	Size	Domain	Notes
HotpotQA	113K Q&A	Wikipedia	Multi-hop; sentence-level supporting facts
2WikiMultiHopQA	192K Q&A	Wikipedia + Wikidata	Explicit reasoning chains; evidence annotations
MuSiQue	25K Q&A	Wikipedia	2–4 hops; designed to prevent reasoning shortcuts
MultiHop-RAG	3.2K Q&A	News	Built for RAG benchmarking; smallest knowledge base

All four are available on the HuggingFace datasets hub. You may use a different benchmark if it satisfies all four of the following: (1) has a train split with (query, positive-passage) pairs for embedder fine-tuning; (2) has an officially published evaluation protocol; (3) is loadable from HuggingFace datasets; (4) has at least 1,000 evaluation samples and 3,000 passages in the knowledge base.

Working size. You do not need to use the full benchmark. A reasonable working size is 3,000–8,000 training pairs for Task 3a and 400–600 held-out questions for evaluation. Document the subsample size and strategy in your README.

Example benchmark justification

Benchmark: HotpotQA. *Justification:* well-documented multi-hop benchmark with official EM/F1 metrics, large train split for embedder fine-tuning, and canonical evaluation dataset for iterative-retrieval agents. *Working size:* 5,000 training pairs; 500 held-out evaluation questions.

If you choose a single-hop benchmark (e.g. Natural Questions, SciFact), the iterative agent in Task 4 may show no improvement or a slight regression. Report this as your finding - it is a valid and informative result.

Task 2. Baseline RAG - Configuration A

2.1 Knowledge base construction

Load the corpus passages from your benchmark. Index them in OpenSearch using HNSW via the provided `docker-compose.yml`. Apply chunking only if the passages are long; most benchmark passages are short enough to index as-is.

Index only the benchmark's official knowledge-base passages. The evaluation questions and their ground-truth answer passages must not appear in any data path used by Task 3a's embedder fine-tuning.

2.2 Embedding and retrieval

Embed all passages with an off-the-shelf small embedder. Implement top- k vector retrieval over the OpenSearch index ($k = 3$ or 5 is a reasonable starting point).

Suggested off-the-shelf embedders

`sentence-transformers/all-MiniLM-L6-v2` (22M params) - fast; good general baseline.
`BAAI/bge-small-en-v1.5` (33M params) - stronger English retrieval; recommended starting point.

2.3 Generator and prompting

Select a generator that fits your available compute. The generator is **prompted only** - do not fine-tune it. Pass the retrieved passages as context and instruct the model to answer from them.

Setting			Suggested generators
CPU-only			Qwen2.5-1.5B / 3B-Instruct via Ollama; Phi-3.5-mini
Laptop	GPU	(≥ 8 GB VRAM)	Llama-3.1-8B / Qwen2.5-7B via Ollama
Free API			Groq (Llama-3.1), Google Gemini Flash, Open-Router free models
Paid API			OpenAI, Anthropic Claude, Google Gemini Pro, Mistral

Document your generator choice and the exact prompt template in your README.

Task 3. Improved Retrieval - Configuration B

3a. Fine-tune the bi-encoder embedder

Fine-tune the embedder from Task 2 on (query, positive-passage) pairs sampled from your benchmark's **training split only**, using a contrastive loss with in-batch negatives.

Suggested framework: `sentence-transformers` with `MultipleNegativesRankingLoss`. Any equivalent framework is acceptable: a custom PyTorch loop, `FlagEmbedding`, or another library of your choice.

Reasonable starting settings: a few thousand training pairs, 1–3 epochs, batch size 16–32. With GPU access, scale up to `bge-base`, `bge-large`, or `e5-large`.

Save the fine-tuned embedder to `models/finetuned_embedder/`. Before and after fine-tuning, compute Recall@5 on a small diagnostic slice (100–200 questions) and report the improvement.

Construct training pairs **exclusively from the benchmark’s train split**. The evaluation questions and their ground-truth passages must never be seen by the embedder during fine-tuning. Once an evaluation example has been seen by any trainable component, the comparison between configurations A, B, and C is invalid. The same rule as “split first, preprocess second”: the split comes first, always.

The embedder must be an open-weight model. Closed API embedders (e.g. OpenAI `text-embedding-3-small`) cannot be fine-tuned and must not be used for Task 3a.

3b. One architectural retrieval technique

Apply **one** of the following techniques on top of your fine-tuned embedder.

Technique	What it does	Suggested libraries
Cross-encoder reranking	Retrieve top-20 with bi-encoder; re-score with cross-encoder; keep top-3	BAAI/bge-reranker-base; FlagEmbedding
Hybrid BM25 + dense	Combine lexical and dense scores via RRF or min-max fusion	OpenSearch hybrid query; rank_bm25; LangChain EnsembleRetriever
HyDE	Generate a hypothetical answer; embed it; retrieve on that embedding	LangChain HypotheticalDocumentEmbedder; hand-rolled
Multi-query expansion	Generate 3–5 query variants; retrieve for each; union results	LangChain MultiQueryRetriever; hand-rolled
Semantic chunking	Replace fixed-size chunks with embedding-similarity-based boundaries	langchain_experimental.SemanticChunker
Parent-document retrieval	Index small chunks; return their larger parent document for generation	LangChain ParentDocumentRetriever

The output of Tasks 3a and 3b together is **Configuration B**.

Example Configuration B

Fine-tuned `bge-small-en-v1.5` with `MultipleNegativesRankingLoss` on 5,000 HotpotQA training pairs, 2 epochs. Recall@5 on diagnostic slice: 0.61 → 0.74. Architectural technique: cross-encoder reranking with `BAAI/bge-reranker-base` over top-20 results, keeping top-3. Final Recall@5: 0.81.

Task 4. Iterative-Retrieve Agent - Configuration C

Wrap Configuration B’s retrieval inside an agent that decides whether and what to retrieve next based on what it has already seen.

4.1 The tool

There is one tool: `retrieve(query) -> list[passage]`. It calls Configuration B’s retrieval pipeline. The agent may call this tool multiple times per question with different queries.

4.2 The loop

Implement a ReAct-style loop:

1. Read the question.
2. Decide whether the current context is sufficient to answer.
3. If not: emit a follow-up search query, call `retrieve`, append results to context. Return to step 2.
4. If yes: emit the final answer and stop.

Enforce a **maximum step budget** (e.g. 5 iterations) to prevent unbounded loops.

4.3 Framework

Generator	Suggested framework
Small models (<2B params)	<code>smolagents</code> (HuggingFace) in <code>CodeAgent</code> mode - code-as-tool-interface is more reliable than JSON at this scale
Mid-size local (7–8B)	<code>smolagents</code> , LangChain agents, LangGraph, or hand-rolled ReAct loop
Strong API models	Any framework, including native function-calling, LangGraph, or hand-rolled

4.4 Required traces

Produce **three worked traces** of your agent answering three different questions from the evaluation set. Save each trace to `src/traces/`. Each trace must include:

- The original question.
- All intermediate *Thought* $\rightarrow$ `retrieve(...)` $\rightarrow$ *Observation* steps.
- The final answer and whether it matches the ground truth.

Include a short paragraph (3–5 sentences) discussing: when did iteration help, when did it not, and did the agent ever loop unproductively?

Example trace (abbreviated)

Question: “What is the population of the country where the director of Inception was born?”
Thought 1: I need to find the director of Inception first.
Action: `retrieve("director of Inception")`
Observation: “Christopher Nolan directed Inception...”
Thought 2: Nolan was born in London, UK. I need the population of the UK.
Action: `retrieve("population of United Kingdom")`
Observation: “...approximately 67 million...”
Final answer: Approximately 67 million. (*Correct. Two retrievals were required.*)

Task 5. Evaluation

5.1 Universal retrieval metrics (mandatory)

Compute the following on the held-out evaluation set for **all three configurations**:

Metric	Definition
Recall@5	Fraction of questions for which the ground-truth passage appears in the top-5 retrieved results
Recall@10	Same at rank 10
MRR@10	Mean Reciprocal Rank: $\frac{1}{ Q } \sum_q \frac{1}{\text{rank of first relevant result}}$, capped at rank 10

Produce a **3-row × 3-column comparison table** (Config A / B / C vs Recall@5 / Recall@10 / MRR@10). Save it to `results/comparison_table.csv`.

Suggested implementations: `pytrec_eval`, `ranx`, or a hand-rolled loop.

5.2 Benchmark-official metrics (recommended)

Report the answer-quality metrics defined in your benchmark’s source paper. These allow your results to be compared against published baselines.

Benchmark	Answer metrics	Evidence metrics
HotpotQA	Answer EM, Answer F1	Supporting-Facts EM/F1, Joint EM/F1
2WikiMultiHopQA	Answer EM, Answer F1	Supporting-Facts EM/F1, Joint EM/F1
MuSiQue	Answer F1	Supporting-Paragraph F1
MultiHop-RAG	Generation accuracy (LLM-judge)	Retrieval Hit@k, MAP@k

5.3 RAGAS deep-dive (optional)

Apply RAGAS (`pip install ragas`) to the best-performing configuration and report: **Faithfulness**, **Context Precision**, and **Context Recall**. RAGAS requires a callable LLM as judge. Free options: Groq free tier (Llama-3.1-70B), Google Gemini Flash free tier, OpenRouter free models, or a local 8B model via Ollama. If none is available, omit this section and state the reason in your README.

5.4 Discussion

Write a short analysis in your README addressing:

- Which configuration performed best and by how much.
- Whether embedder fine-tuning helped, and how much the architectural technique added on top.
- Whether iterative retrieval helped or added noise.
- A brief error analysis on 3–5 example questions where one configuration consistently won or lost.

Deliverables

Submit a GitHub repository. The three configurations must be **individually runnable**. Evaluation logic must be **isolated** from pipeline logic. A reader must be able to clone the repository and reproduce your results without modifying code.

File / Folder	Content
src/baseline_rag.py	Configuration A: off-the-shelf embedder + top- <i>k</i> retrieval
src/improved_retrieval.py	Configuration B: fine-tuned embedder + architectural technique
src/agent.py	Configuration C: iterative-retrieval agent
src/evaluate.py	Recall@5, Recall@10, MRR@10 across all three configurations
src/traces/	Three saved agent execution traces (Task 4.4)
data/corpus.jsonl	Indexed knowledge-base passages
data/train_pairs.jsonl	(query, positive-passage) pairs used for fine-tuning
data/eval_set.jsonl	Held-out evaluation questions and ground-truth answers
models/finetuned_embedder/	Saved bi-encoder after contrastive fine-tuning
results/comparison_table.csv	Metric values for all three configurations
results/ragas_scores.json	RAGAS scores on best configuration (optional)
docker-compose.yml	OpenSearch instance
requirements.txt	All dependencies with versions
README.md	See requirements below

Generator inference and RAGAS calls are expensive in time, money, or rate-limit quota. Cache all outputs to disk so that re-running your evaluation script does not repeat the same API or inference calls. Label cached files clearly.

Set `random_state=42` (or equivalent) wherever seeds are accepted: train/test splits, embedder training, agent sampling. Set `temperature=0` for the generator wherever supported. Non-deterministic evaluation results cannot be reproduced or compared.

README Requirements

Your `README.md` must contain the following eight sections:

1. **Benchmark choice** - which benchmark and why; if Option B, demonstrate it satisfies all four rules.
2. **Compute and model setup** - hardware used and generator model, with a short justification of the choice.
3. **Configuration A** - embedder, chunking strategy, retrieval parameters, exact prompt template.
4. **Configuration B** - fine-tuning details (number of pairs, loss, training parameters, before/after Recall@*k* on diagnostic slice); architectural technique chosen and why.
5. **Configuration C** - framework chosen, prompt template, summary of the three worked traces.
6. **Evaluation results** - comparison table; benchmark-official metrics if reported; RAGAS scores if Task 5.3 was attempted.
7. **Findings and error analysis** - what helped, what did not, 3–5 example questions discussed in detail.
8. **Installation and execution** - step-by-step commands to clone the repository, install dependencies, start OpenSearch, run the pipeline, and reproduce the results table.